The Theory of Computing

Countable and Uncountable Sets

Ueber eine Eigenschaft des Inbegriffes aller reellen algebraischen Zahlen.

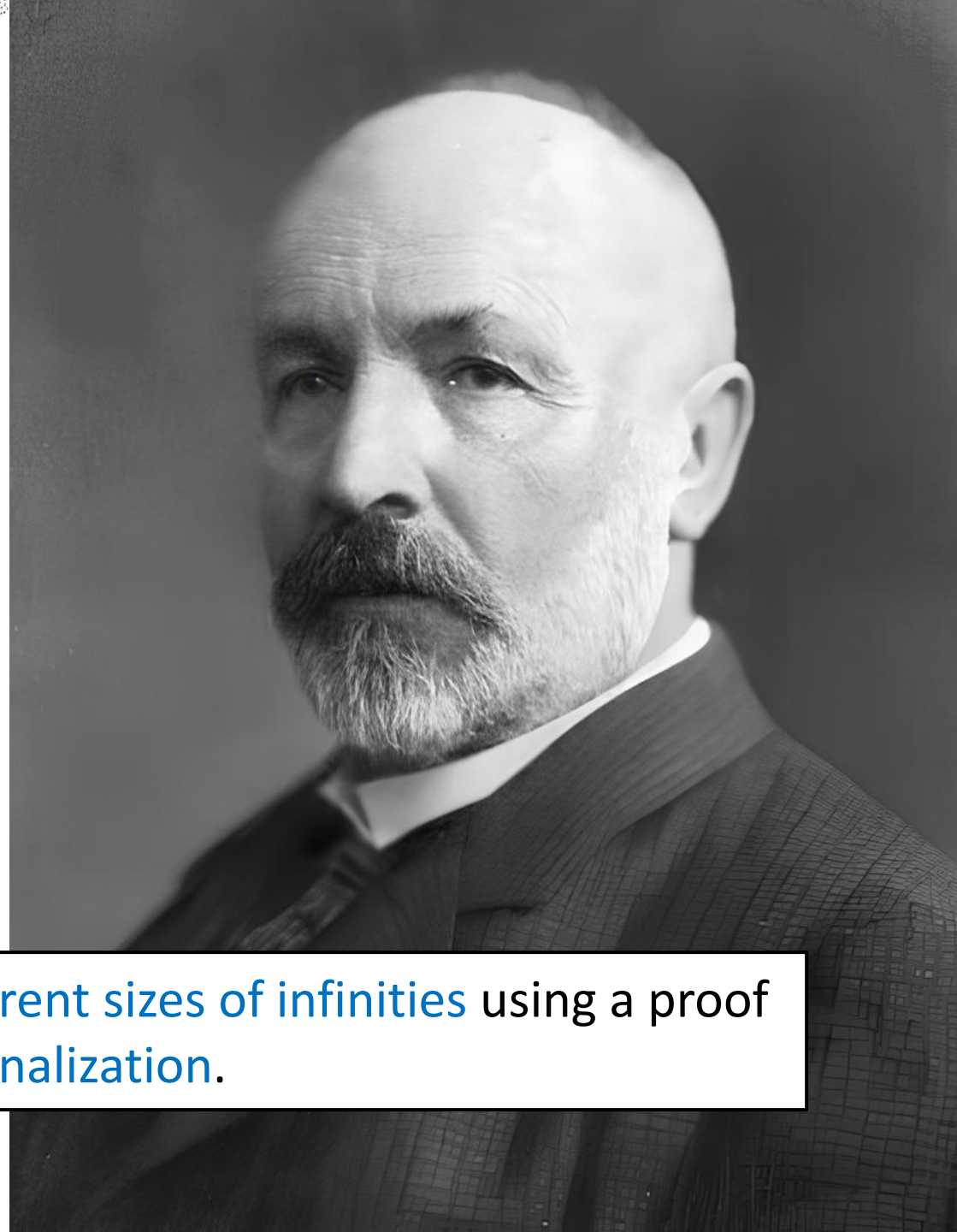
(Von Herrn *Cantor* in Halle a. S.)

Unter einer reellen algebraischen Zahl wird allgemein eine reelle Zahlgrösse ω verstanden, welche einer nicht identischen Gleichung von der Form genügt:

$$(1.) \quad a_0 \omega^n + a_1 \omega^{n-1} + \cdots + a_n = 0,$$

wo $n, a_0, a_1, \cdots a_n$ ganze Zahlen sind; wir können uns hierbei die Zahlen n und a_0 positiv, die Coefficienten $a_0, a_1, \cdots a_n$ ohne gemeinschaftlichen Theiler und die Gleichung (1.) irreductibel denken; mit diesen Festsetzungen wird erreicht, dass nach den bekannten Grundsätzen der Arithmetik und Algebra die Gleichung (1.), welcher eine reelle algebraische Zahl genügt, eine völlig bestimmte ist; umgekehrt gehören bekanntlich zu einer Gleichung

geht, eine solche Beschaffenheit, dass in jeder Nähe irgend einer gedachten Zahl α unendlich viele Zahlen aus (ω) liegen; um so auffallen-



In 1874, Georg Cantor showed there were different sizes of infinities using a proof technique called diagonalization.

If we take the odd natural numbers and add them to the even natural numbers, we get all the natural numbers. Each of these sets is of infinite size.

$$\infty + \infty = \infty$$

Are the natural numbers bigger than the even or odd numbers?

Cantor helped us see these sets were all the same size using a bijection:

Natural	0	1	2	3	4	5	6	7	8	…
Even	0	2	4	6	8	10	12	14	16	…
Odd	1	3	5	7	9	11	13	15	17	…

Since, in a bijection, every member of one set is paired with a member of the other set, we know they must be of the same size.

Natural	0	1	2	3	4	5	6	7	8	…
Even	0	2	4	6	8	10	12	14	16	…
Odd	1	3	5	7	9	11	13	15	17	…

Definition: An infinite set is countable if it has the same size as the natural numbers.

This means if we can list all the elements in a single long list, the set is countable.

Using this idea, we can show that the set of integers is countable.

Natural	0	1	2	3	4	5	6	7	8	…
Integers	0	1	-1	2	-2	3	-3	4	-4	…

Consider the set of all pairs of natural numbers.

$$(n, m) \in \mathbb{N} \times \mathbb{N}$$

$$\infty \cdot \infty = \infty$$

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)	(0,8)	(0,9)	(0,10)	(0,11)	(0,12)	(0,13
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)	(1,8)	(1,9)	(1,10)	(1,11)	(1,12)	(1,13
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)	(2,8)	(2,9)	(2,10)	(2,11)	(2,12)	(2,13
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)	(3,8)	(3,9)	(3,10)	(3,11)	(3,12)	(3,13
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)	(4,5)	(4,6)	(4,7)	(4,8)	(4,9)	(4,10)	(4,11)	(4,12)	(4,13
(5,0)	(5,1)	(5,2)	(5,3)	(5,4)	(5,5)	(5,6)	(5,7)	(5,8)	(5,9)	(5,10)	(5,11)	(5,12)	(5,13
(6,0)	(6,1)	(6,2)	(6,3)	(6,4)	(6,5)	(6,6)	(6,7)	(6,8)	(6,9)	(6,10)	(6,11)	(6,12)	(6,13
(7,0)	(7,1)	(7,2)	(7,3)	(7,4)	(7,5)	(7,6)	(7,7)	(7,8)	(7,9)	(7,10)	(7,11)	(7,12)	(7,13
(8,0)	(8,1)	(8,2)	(8,3)	(8,4)	(8,5)	(8,6)	(8,7)	(8,8)	(8,9)	(8,10)	(8,11)	(8,12)	(8,13
(9,0)	(9,1)	(9,2)	(9,3)	(9,4)	(9,5)	(9,6)	(9,7)	(9,8)	(9,9)	(9,10)	(9,11)	(9,12)	(9,13
(10,0)	(10,1)	(10,2)	(10,3)	(10,4)	(10,5)	(10,6)	(10,7)	(10,8)	(10,9)	(10,10)	(10,11)	(10,12)	(10,13
(11,0)	(11,1)	(11,2)	(11,3)	(11,4)	(11,5)	(11,6)	(11,7)	(11,8)	(11,9)	(11,10)	(11,11)	(11,12)	(11,13
(12,0)	(12,1)	(12,2)	(12,3)	(12,4)	(12,5)	(12,6)	(12,7)	(12,8)	(12,9)	(12,10)	(12,11)	(12,12)	(12,13
(13,0)	(13,1)	(13,2)	(13,3)	(13,4)	(13,5)	(13,6)	(13,7)	(13,8)	(13,9)	(13,10)	(13,11)	(13,12)	(13,13
(14,0)													(14,13
(15,0)													(15,13
(16,0)													(16,13
(17,0)	(17,1)	(17,2)	(17,3)	(17,4)	(17,5)	(17,6)	(17,7)	(17,8)	(17,9)	(17,10)	(17,11)	(17,12)	(17,13

We can list all the pairs in a 2-dimensional table, but can we list them all in a single 1-dimensional list?

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)	(0,8)	(0,9)	(0,10)	(0,11)	(0,12)	(0,13
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)	(1,8)	(1,9)	(1,10)	(1,11)	(1,12)	(1,13
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)	(2,8)	(2,9)	(2,10)	(2,11)	(2,12)	(2,13
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)	(3,8)	(3,9)	(3,10)	(3,11)	(3,12)	(3,13
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)	(4,5)	(4,6)	(4,7)	(4,8)	(4,9)	(4,10)	(4,11)	(4,12)	(4,13
(5,0)	(5,1)	(5,2)	(5,3)	(5,4)	(5,5)	(5,6)	(5,7)	(5,8)	(5,9)	(5,10)	(5,11)	(5,12)	(5,13
(6,0)	(6,1)	(6,2)	(6,3)	(6,4)	(6,5)	(6,6)	(6,7)	(6,8)	(6,9)	(6,10)	(6,11)	(6,12)	(6,13
(7,0)	(7,1)	(7,2)	(7,3)	(7,4)	(7,5)	(7,6)	(7,7)	(7,8)	(7,9)	(7,10)	(7,11)	(7,12)	(7,13
(8,0)	(8,1)	(8,2)	(8,3)	(8,4)	(8,5)	(8,6)	(8,7)	(8,8)	(8,9)	(8,10)	(8,11)	(8,12)	(8,13
(9,0)	(9,1)	(9,2)	(9,3)	(9,4)	(9,5)	(9,6)	(9,7)	(9,8)	(9,9)	(9,10)	(9,11)	(9,12)	(9,13
(10,0)	(10,1)	(10,2)	(10,3)	(10,4)	(10,5)	(10,6)	(10,7)	(10,8)	(10,9)	(10,10)	(10,11)	(10,12)	(10,13
(11,0)	(11,1)	(11,2)	(11,3)	(11,4)	(11,5)	(11,6)	(11,7)	(11,8)	(11,9)	(11,10)	(11,11)	(11,12)	(11,13
(12,0)	(12,1)	(12,2)	(12,3)	(12,4)	(12,5)	(12,6)	(12,7)	(12,8)	(12,9)	(12,10)	(12,11)	(12,12)	(12,13
(13,0)	(13,1)	(13,2)	(13,3)	(13,4)	(13,5)	(13,6)	(13,7)	(13,8)	(13,9)	(13,10)	(13,11)	(13,12)	(13,13
(14,0)													(14,13
(15,0)													(15,13
(16,0)													(16,13
(17,0)	(17,1)	(17,2)	(17,3)	(17,4)	(17,5)	(17,6)	(17,7)	(17,8)	(17,9)	(17,10)	(17,11)	(17,12)	(17,13

Naively trying to go row-by-row, or column-by-column, will not reach every pair.

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)	(0,8)	(0,9)	(0,10)	(0,11)	(0,12)	(0,13
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)	(1,8)	(1,9)	(1,10)	(1,11)	(1,12)	(1,13
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)	(2,8)	(2,9)	(2,10)	(2,11)	(2,12)	(2,13
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)	(3,8)	(3,9)	(3,10)	(3,11)	(3,12)	(3,13
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)	(4,5)	(4,6)	(4,7)	(4,8)	(4,9)	(4,10)	(4,11)	(4,12)	(4,13
(5,0)	(5,1)	(5,2)	(5,3)	(5,4)	(5,5)	(5,6)	(5,7)	(5,8)	(5,9)	(5,10)	(5,11)	(5,12)	(5,13
(6,0)	(6,1)	(6,2)	(6,3)	(6,4)	(6,5)	(6,6)	(6,7)	(6,8)	(6,9)	(6,10)	(6,11)	(6,12)	(6,13
(7,0)	(7,1)	(7,2)	(7,3)	(7,4)	(7,5)	(7,6)	(7,7)	(7,8)	(7,9)	(7,10)	(7,11)	(7,12)	(7,13
(8,0)	(8,1)	(8,2)	(8,3)	(8,4)	(8,5)	(8,6)	(8,7)	(8,8)	(8,9)	(8,10)	(8,11)	(8,12)	(8,13
(9,0)	(9,1)	(9,2)	(9,3)	(9,4)	(9,5)	(9,6)	(9,7)	(9,8)	(9,9)	(9,10)	(9,11)	(9,12)	(9,13
(10,0)	(10,1)	(10,2)	(10,3)	(10,4)	(10,5)	(10,6)	(10,7)	(10,8)	(10,9)	(10,10)	(10,11)	(10,12)	(10,13
(11,0)	(11,1)	(11,2)	(11,3)	(11,4)	(11,5)	(11,6)	(11,7)	(11,8)	(11,9)	(11,10)	(11,11)	(11,12)	(11,13
(12,0)	(12,1)	(12,2)	(12,3)	(12,4)	(12,5)	(12,6)	(12,7)	(12,8)	(12,9)	(12,10)	(12,11)	(12,12)	(12,13
(13,0)	(13,1)	(13,2)	(13,3)	(13,4)	(13,5)	(13,6)	(13,7)	(13,8)	(13,9)	(13,10)	(13,11)	(13,12)	(13,13
(14,0)													(14,13
(15,0)													(15,13
(16,0)													(16,13
(17,0)	(17,1)	(17,2)	(17,3)	(17,4)	(17,5)	(17,6)	(17,7)	(17,8)	(17,9)	(17,10)	(17,11)	(17,12)	(17,13

However, if we are a bit clever, and go diagonal-by-diagonal, we can list them all.

So, the set of all pairs of natural numbers is countable.

$$(n, m) \in \mathbb{N} \times \mathbb{N}$$

Since every positive rational number can be expressed as $\frac{p}{q}$ for a pair of natural numbers p and q the set of rational numbers $\mathbb{Q}$ is also countable.

Now, consider the real numbers $\mathbb{R}$. Can we put them in a single list?

Spoilers: No.

In the natural numbers, if you select any two numbers n and m then there are only a finite number of numbers between them.

In the real numbers, if you select any two numbers n and m then there are still infinite numbers between them. We call this an interval on the real numbers.

$$[n, m]$$
$$(n, m)$$
$$(n, m]$$
$$[n, m)$$

Theorem: The set of real numbers is uncountable.

We will focus on this interval of real numbers between 0 and 1.

$$[0,1)$$

If is uncountable, then so is the entire range of real numbers $(-\infty, \infty)$.

$$[0,1)$$

Every number on this interval can be written as an infinite string of bits after the decimal point.

$$0.b_1 b_2 b_3 \ldots$$

Assume for the sake of contradiction that the numbers on the interval $[0,1)$ are countable.

So, the numbers on the interval $[0,1)$ can be placed on a countable list r_1, r_2, r_3 …

$$r_1 = 0.01001010101010\ldots$$
$$r_2 = 0.10111010101010\ldots$$
$$r_3 = 0.11100010100100\ldots$$
$$r_4 = 0.10100101010100\ldots$$
$$\vdots$$

Now define a special number

$$d = 0.d_1 d_2 d_3 \ldots$$

where $d_i = !\, r_{i,i}$ and where $r_{i,i}$ is the ith bit of r_i.

$$d = 0.1101 \ldots$$

$$\begin{aligned} r_1 &= 0.\mathbf{0}100101010101 \ldots \\ r_2 &= 0.1\mathbf{0}11101010101 \ldots \\ r_3 &= 0.11\mathbf{1}0001010010 \ldots \\ r_4 &= 0.101\mathbf{0}010101010 \ldots \\ &\vdots \end{aligned}$$

We say that d diagonalizes against our list of numbers to create a number that is not on our list.

$$d = 0.1101\ldots$$

$$\begin{aligned} r_1 &= 0.\mathbf{0}100101010101\ldots \\ r_2 &= 0.1\mathbf{0}11101010101\ldots \\ r_3 &= 0.11\mathbf{1}0001010010\ldots \\ r_4 &= 0.101\mathbf{0}010101010\ldots \\ &\vdots \end{aligned}$$

We know d is not on our list because $d \neq r_i$ since we know that $d_i \neq r_{i,i}$.

$$d = 0.1101\ldots$$

$$\begin{aligned} r_1 &= 0.0100101010101\ldots \\ r_2 &= 0.1011101010101\ldots \\ r_3 &= 0.1110001010010\ldots \\ r_4 &= 0.1010010101010\ldots \\ &\vdots \end{aligned}$$

This contradicts our assumption that the list had every number in the interval $[0,1)$.

So, the set of real numbers is uncountable.

As a result, we have these three theorems:

$$|\mathbb{N}| = \infty$$
$$|\mathbb{R}| = \infty$$
$$|\mathbb{N}| < |\mathbb{R}|$$

To accommodate different sizes of ∞ we use a new aleph notation.

Definition: $\aleph_0$, called aleph-null, is the size of the natural numbers.

Definition: $\aleph_1$, called aleph-one, is the size of the next largest infinity.

Continuum Hypothesis: $\aleph_1$, called aleph-one, is the size of the real numbers.

Theorem: The set of finite binary strings is countable.

Theorem: The set of infinite binary strings is uncountable.

Theorem: The set of Turing machines is countable.

Proof: Recall that a Turing machine can be encoded as a finite length binary string.

Since the set of finite length binary strings is countable, the set of Turing machines is also countable.

Theorem: The set of languages is uncountable.

Proof: A language is a set of finite length binary strings.

Since the set of finite length binary strings is countable, we can put them on a list, say in order of size.

Now for each potential string in the language we can use a 0 to represent not being in the language and a 1 to represent being in the language.

This means every infinite length binary string corresponds to a distinct language.

The set of infinite length binary strings is uncountable, so the set of languages must also be uncountable.

Theorem: The set of languages is uncountable.

Proof: A language is a set of finite length binary strings.

Since the set of finite length binary strings is countable, we can put them on a list, say in order of size.

Now for each potential string in the language we can use a 0 to represent not being in the language and a 1 to represent being in the language.

This means every infinite length binary string corresponds to a distinct language.

The set of infinite length binary strings is uncountable, so the set of languages must also be uncountable.

Theorem: The set of languages is uncountable.

Proof: A language is a set of finite length binary strings.

Since the set of finite length binary strings is countable, we can put them on a list, say in order of size.

Now for each potential string in the language we can use a 0 to represent not being in the language and a 1 to represent being in the language.

This means every infinite length binary string corresponds to a distinct language.

The set of infinite length binary strings is uncountable, so the set of languages must also be uncountable.

Theorem: The set of languages is uncountable.

Proof: A language is a set of finite length binary strings.

Since the set of finite length binary strings is countable, we can put them on a list, say in order of size.

Now for each potential string in the language we can use a 0 to represent not being in the language and a 1 to represent being in the language.

This means every infinite length binary string corresponds to a distinct language.

The set of infinite length binary strings is uncountable, so the set of languages must also be uncountable.

Theorem: The set of languages is uncountable.

Proof: A language is a set of finite length binary strings.

Since the set of finite length binary strings is countable, we can put them on a list, say in order of size.

Now for each potential string in the language we can use a 0 to represent not being in the language and a 1 to represent being in the language.

This means every infinite length binary string corresponds to a distinct language.

The set of infinite length binary strings is uncountable, so the set of languages must also be uncountable.

Corollary: There is a language that is not Turing recognizable (or Turing decidable).

The Theory of Computing

An Undecidable Language

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

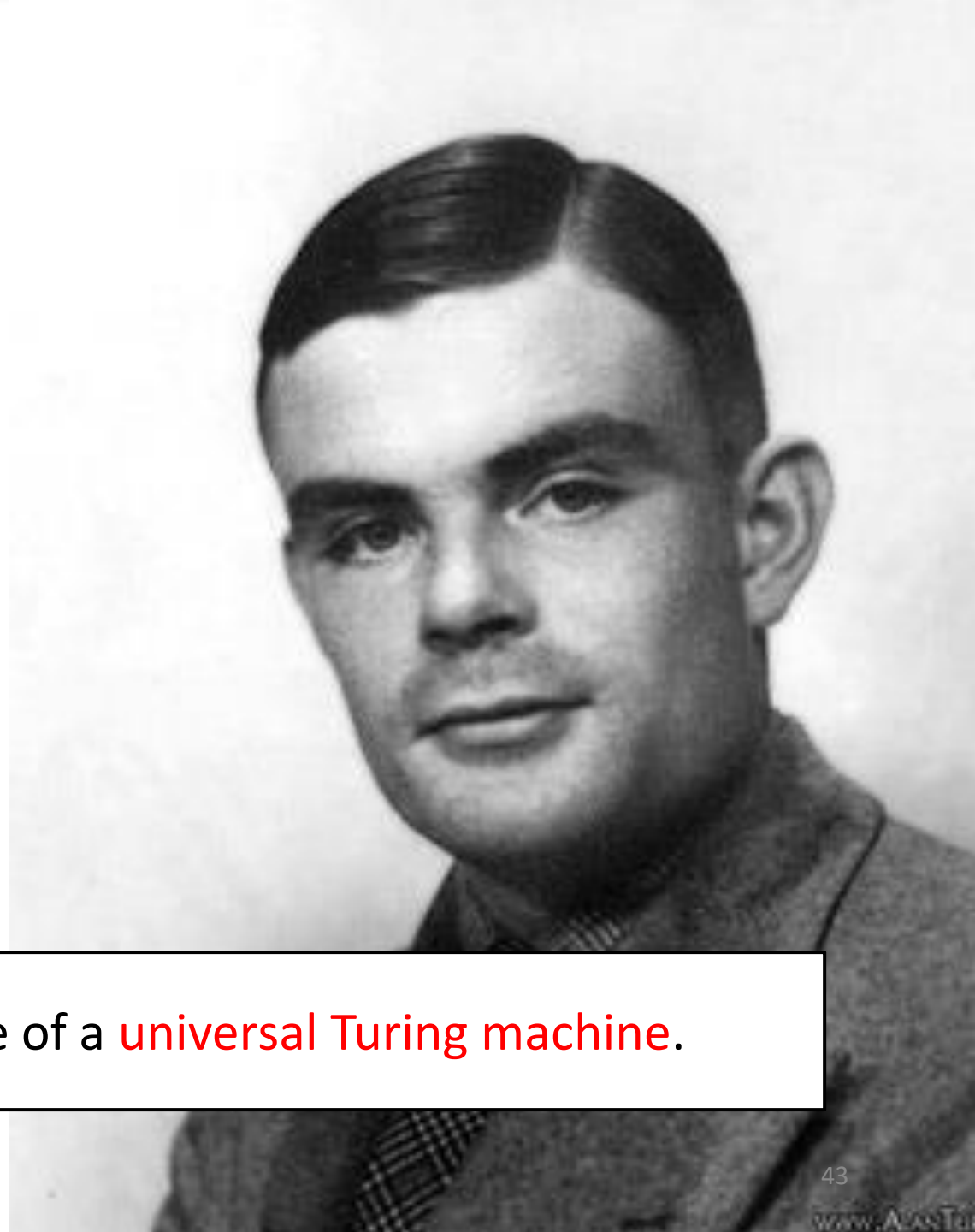
By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The "computable" numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers

putable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

In §§ 9, 10 I give some arguments with the intention of showing that the



In 1936, Alan Turing showed the existence of a universal Turing machine.

$$A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM that accepts the string } w\}$$

Theorem: A_{TM} is a recognizable language.

Proof idea: We will construct a TM U_{TM} that recognizes A_{TM}.

Let U_{TM} = "On input $\langle M, w\rangle$, where M is a TM and w is a string:

1. Simulate M on input w.
2. If M ever enters an accept state, accept.
3. If M ever enters a reject state, reject."

Simulation of an arbitrary Turing-machine is usually carried out on a multi-tape Turing machine, as we did when simulating non-deterministic Turing machines.

The machine U_{TM} is called the universal Turing machine as it can simulate any Turing machine, including itself.

The universal Turing machine influenced the development of stored programs over coding things in hardware (i.e., “hard-coding”).

Notice that U_{TM} recognizes A_{TM}, but it does not decide it, as it accepts all strings in A_{TM}, but does not reject all strings not in A_{TM}.

This raises the question of whether A_{TM} is decidable.

Spoilers: It is not.

To show A_{TM} is not decidable Turing used diagonalization as Cantor did for the real numbers.

$$A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM that accepts the string } w\}$$

Theorem: A_{TM} is not decidable.

Proof: Assume that A_{TM} is decidable. Then there must exist some Turing machine H such that H decides A_{TM}.

Note that when H is run on $\langle M, w\rangle$ will accept whenever M accepts w and it will reject whenever M rejects w or loops forever.

$$A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM that accepts the string } w\}$$

Theorem: A_{TM} is not decidable.

Proof (continued): Consider the machine D:

"On input $\langle M\rangle$ where M is a TM:

1. Run H on input $\langle M, \langle M\rangle\rangle$.
2. If H accepts, reject.
3. If H rejects, accept."

$$A_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM that accepts the string } w\}$$

Theorem: A_{TM} is not decidable.

Proof (continued): If we run D on $\langle D\rangle$ we get that:

D accepts $\langle D\rangle$ if D does not accept $\langle D\rangle$.

D rejects $\langle D\rangle$ if D accepts $\langle D\rangle$.

This is a contradiction.

D cannot exist. So, our assumption that H exists was false.

So A_{TM} is not decidable. □

Theorem: A language L is Turing decidable if and only if L and $\bar{L}$ are Turing recognizable.

Proof: Assume L is Turing decidable.

Then there must be a Turing machine M that decides L.

Note that M also recognizes L, so L is recognizable.

Theorem: A language L is Turing decidable if and only if L and $\bar{L}$ are Turing recognizable.

Proof: We can construct a Turing machine $\bar{M}$ out of M by switching its accept and reject states.

Since M decides L, $\bar{M}$ decides $\bar{L}$ so $\bar{L}$ is Turing recognizable.

So, the first branch has been proved, if L is Turing decidable then L and $\bar{L}$ are Turing recognizable.

Theorem: A language L is Turing decidable if and only if L and $\bar{L}$ are Turing recognizable.

Proof: Assume that L and $\bar{L}$ are Turing recognizable.

Then there must be machines M and $\bar{M}$ that recognize them, respectively.

We can construct a Turing machine M_1 that simulates M and $\bar{M}$ one step at a time in parallel.

One of these machines is guaranteed to halt and accept.

If M accepts, accept. If $\bar{M}$ accepts, reject.

M_1 halts on all inputs and decides L. □

Theorem: $\overline{A_{TM}}$ is unrecognizable.

Proof: Assume $\overline{A_{TM}}$ is recognizable.

We know A_{TM} is recognizable.

By the last theorem A_{TM} must be decidable.

We have previously shown that A_{TM} is not decidable.

This is a contradiction.

So $\overline{A_{TM}}$ must not be recognizable. □

The Theory of Computing

Turing Reductions

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ is a Turing machine that halts on } w\}$$

This language, called the Halting Problem, is undecidable.

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ is a Turing machine that halts on } w\}$$

We can prove this directly, using diagonalization, or indirectly, using a Turing-reduction.

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ is a Turing machine that halts on } w\}$$

Theorem: $HALT_{TM}$ is undecidable.

Proof: Assume that $HALT_{TM}$ is decidable and that H decides $HALT_{TM}$.

We will build the machine M_1:

"On input $\langle M, w\rangle$ where M is a TM and w is a string:

1. Run H on $\langle M, w\rangle$.
2. If H rejects, reject.
3. If H accepts run M on w.
4. If M accepts, accept.
5. If M rejects, reject."

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ is a Turing machine that halts on } w\}$$

Theorem: $HALT_{TM}$ is undecidable.

Proof: Assume that $HALT_{TM}$ is decidable and that H decides $HALT_{TM}$.

We will build the machine M_1:

"On input $\langle M, w\rangle$ where M is a TM and w is a string:

1. Run H on $\langle M, w\rangle$.
2. If H rejects, reject.
3. If H accepts run M on w.
4. If M accepts, accept.
5. If M rejects, reject."

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ is a Turing machine that halts on } w\}$$

Theorem: $HALT_{TM}$ is undecidable.

Proof (continued): "On input $\langle M, w\rangle$ where M is a TM and w is a string:

1. Run H on $\langle M, w\rangle$.
2. If H rejects, reject.
3. If H accepts run M on w.
4. If M accepts, accept.
5. If M rejects, reject."

We can see that M_1 halts on every input since H halts on every input and we only run M on w if we know that M halts on w. This means that M_1 decides A_{TM}.

A_{TM} is undecidable so $HALT_{TM}$ is undecidable. □

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ is a Turing machine that halts on } w\}$$

Theorem: $HALT_{TM}$ is undecidable.

Proof (continued): "On input $\langle M, w\rangle$ where M is a TM and w is a string:

1. Run H on $\langle M, w\rangle$.
2. If H rejects, reject.
3. If H accepts run M on w.
4. If M accepts, accept.
5. If M rejects, reject."

We can see that M_1 halts on every input since H halts on every input and we only run M on w if we know that M halts on w. This means that M_1 decides A_{TM}.

A_{TM} is undecidable so $HALT_{TM}$ is undecidable. □

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing machine that accepts no strings}\}$$

Let's show this language is undecidable.

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a Turing machine that accepts no strings}\}$$

Theorem: E_{TM} is undecidable.

Proof: Assume that E_{TM} is decidable and that H decides E_{TM}.

We will build the machine M_1 using H:

Let M_1: "On input $\langle M, w \rangle$ where M is a TM and w is a string:

1. Construct the machine M_2:

   "On input x:

   1. Run M on w.
   2. If M accepts, accept."

2. Run H on $\langle M_2 \rangle$.
3. If H rejects, accept.
4. If H accept, reject."

Note that M_2 will run M on w for every input x.

So, $L(M_2) = \Sigma^*$, if M accepts w and $L(M_2) = \emptyset$, otherwise.

So, if we run H on M_2, it will reject if M accepts w and accept, otherwise.

So, M_1 accepts $\langle M, w \rangle$ if M accepts w, and rejects, otherwise.

So, M_1 decides A_{TM}.

This is a contradiction.

So, E_{TM} is undecidable. □

Let M_1: "On input $\langle M, w \rangle$ where M is a TM and w is a string:

1. Construct the machine M_2:

   "On input x:

   1. Run M on w.
   2. If M accepts, accept."

2. Run H on $\langle M_2 \rangle$.
3. If H rejects, accept.
4. If H accept, reject."

Note that M_2 will run M on w for every input x.

So, $L(M_2) = \Sigma^*$, if M accepts w and $L(M_2) = \emptyset$, otherwise.

So, if we run H on M_2, it will reject if M accepts w and accept, otherwise.

So, M_1 accepts $\langle M, w \rangle$ if M accepts w, and rejects, otherwise.

So, M_1 decides A_{TM}.

This is a contradiction.

So, E_{TM} is undecidable. □

Let M_1: "On input $\langle M, w \rangle$ where M is a TM and w is a string:

1. Construct the machine M_2:

   "On input x:

   1. Run M on w.
   2. If M accepts, accept."

2. Run H on $\langle M_2 \rangle$.
3. If H rejects, accept.
4. If H accept, reject."

Note that M_2 will run M on w for every input x.

So, $L(M_2) = \Sigma^*$, if M accepts w and $L(M_2) = \emptyset$, otherwise.

So, if we run H on M_2, it will reject if M accepts w and accept, otherwise.

So, M_1 accepts $\langle M, w \rangle$ if M accepts w, and rejects, otherwise.

So, M_1 decides A_{TM}.

This is a contradiction.

So, E_{TM} is undecidable. □

Let M_1: "On input $\langle M, w \rangle$ where M is a TM and w is a string:

1. Construct the machine M_2:

   "On input x:

   1. Run M on w.
   2. If M accepts, accept."

2. Run H on $\langle M_2 \rangle$.
3. If H rejects, accept.
4. If H accept, reject."

Note that M_2 will run M on w for every input x.

So, $L(M_2) = \Sigma^*$, if M accepts w and $L(M_2) = \emptyset$, otherwise.

So, if we run H on M_2, it will reject if M accepts w and accept, otherwise.

So, M_1 accepts $\langle M, w \rangle$ if M accepts w, and rejects, otherwise.

So, M_1 decides A_{TM}.

This is a contradiction.

So, E_{TM} is undecidable. □

Let M_1: "On input $\langle M, w \rangle$ where M is a TM and w is a string:

1. Construct the machine M_2:

   "On input x:

   1. Run M on w.
   2. If M accepts, accept."

2. Run H on $\langle M_2 \rangle$.
3. If H rejects, accept.
4. If H accept, reject."

Note that M_2 will run M on w for every input x.

So, $L(M_2) = \Sigma^*$, if M accepts w and $L(M_2) = \emptyset$, otherwise.

So, if we run H on M_2, it will reject if M accepts w and accept, otherwise.

So, M_1 accepts $\langle M, w \rangle$ if M accepts w, and rejects, otherwise.

So, M_1 decides A_{TM}.

This is a contradiction.

So, E_{TM} is undecidable. □

Let M_1: "On input $\langle M, w \rangle$ where M is a TM and w is a string:

1. Construct the machine M_2:

   "On input x:

   1. Run M on w.
   2. If M accepts, accept."

2. Run H on $\langle M_2 \rangle$.
3. If H rejects, accept.
4. If H accept, reject."

Note that M_2 will run M on w for every input x.

So, $L(M_2) = \Sigma^*$, if M accepts w and $L(M_2) = \emptyset$, otherwise.

So, if we run H on M_2, it will reject if M accepts w and accept, otherwise.

So, M_1 accepts $\langle M, w \rangle$ if M accepts w, and rejects, otherwise.

So, M_1 decides A_{TM}.

This is a contradiction.

So, E_{TM} is undecidable. □

Let M_1: "On input $\langle M, w \rangle$ where M is a TM and w is a string:

1. Construct the machine M_2:

   "On input x:

   1. Run M on w.
   2. If M accepts, accept."

2. Run H on $\langle M_2 \rangle$.
3. If H rejects, accept.
4. If H accept, reject."

Note that M_2 will run M on w for every input x.

So, $L(M_2) = \Sigma^*$, if M accepts w and $L(M_2) = \emptyset$, otherwise.

So, if we run H on M_2, it will reject if M accepts w and accept, otherwise.

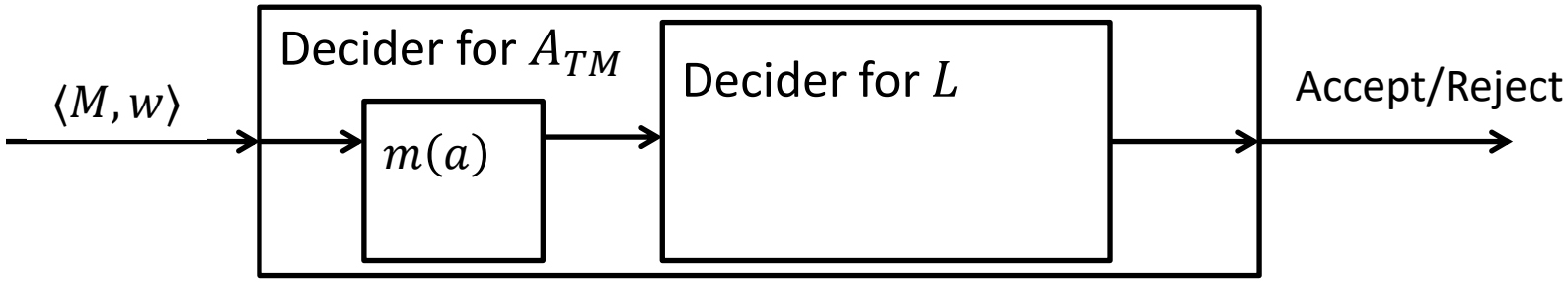
So, M_1 accepts $\langle M, w \rangle$ if M accepts w, and rejects, otherwise.

So, M_1 decides A_{TM}.

This is a contradiction.

So, E_{TM} is undecidable. □

In these proofs we have used Turing-reductions to compare languages.



Definition: A language L_1 reduces to L_2, denoted $L_1 \leq L_2$, if there exists a mapping $m(w)$ such that:

- If $w \in L_1$ then $m(w) \in L_2$.
- If $w \notin L_1$ then $m(w) \notin L_2$.

Definition: A language L_1 Turing-reduces to L_2, denoted $L_1 \leq_T L_2$, if there exists a mapping $m(w)$ such that:

- If $w \in L_1$ then $m(w) \in L_2$.
- If $w \notin L_1$ then $m(w) \notin L_2$.
- $m(w)$ is a Turing-computable mapping.

Definition: A language L_1 Cook-reduces to L_2, denoted $L_1 \leq_P L_2$, if there exists a mapping $m(w)$ such that:

- If $w \in L_1$ then $m(w) \in L_2$.
- If $w \notin L_1$ then $m(w) \notin L_2$.
- $m(w)$ is a polynomial-time Turing-computable mapping.

Theorem: If $L_1 \leq_T L_2$ and L_1 is undecidable then L_2 is undecidable.

Proof: Assume that $L_1 \leq_T L_2$ and that L_1 is undecidable. Assume L_2 is decidable.

Let M_2 be a machine that decides L_2.

Construct M:

"On input w:

1. Compute $m(w)$.
2. Run M_2 on $m(w)$.
3. If M_2 accepts, accept.
4. If M_2 rejects, reject."

M decides L_1 so L_1 is decidable. This is a contradiction. So L_2 must be undecidable. □

Theorem: If $L_1 \leq_T L_2$ and L_1 is undecidable then L_2 is undecidable.

Proof: Assume that $L_1 \leq_T L_2$ and that L_1 is undecidable. Assume L_2 is decidable.

Let M_2 be a machine that decides L_2.

Construct M:

"On input w:

1. Compute $m(w)$.
2. Run M_2 on $m(w)$.
3. If M_2 accepts, accept.
4. If M_2 rejects, reject."

M decides L_1 so L_1 is decidable. This is a contradiction. So L_2 must be undecidable. □

Theorem: If $L_1 \leq_T L_2$ and L_1 is undecidable then L_2 is undecidable.

Proof: Assume that $L_1 \leq_T L_2$ and that L_1 is undecidable. Assume L_2 is decidable.

Let M_2 be a machine that decides L_2.

Construct M:

"On input w:

1. Compute $m(w)$.
2. Run M_2 on $m(w)$.
3. If M_2 accepts, accept.
4. If M_2 rejects, reject."

M decides L_1 so L_1 is decidable. This is a contradiction. So L_2 must be undecidable. □

Exercises

$ALL_{TM} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \Sigma^*\}$

$EQ_{TM} = \{\langle M_1, M_2 \rangle \mid M_1 \text{and } M_2 \text{ are TMs and } L(M_1) = L(M_2)\}$

$TM_\epsilon = \{\langle M \rangle \mid M \text{ is a TM and } \epsilon \in L(M)\}$

$TM_x = \{\langle M \rangle \mid M \text{ is a TM and } x \in L(M)\}$ for any string x

$TM_L = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = L\}$ for any Turing-decidable language L

$REG_{TM} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is regular}\}$